

Python Assignment-24

1. What is a lambda function in Python? How is it different from a normal function?

Answer :

A **lambda function** is a **small anonymous(Without Name) function** created using the lambda keyword. It can have any number of arguments but **only one expression**.

Syntax :

lambda arguments: expression

Normal Example :

```
def Addition(a, b):  
    return a + b  
  
print(Addition(10, 20))
```

Lambda Example :

```
Square = lambda x: x * x  
  
print(Square(5))
```

Difference Normal vs Lambada

Normal Function	Lambda Function
Created using def	Created using lambda
Has a function name	Anonymous (no name unless assigned to a variable)
Can contain multiple statements	Can contain only one expression
Can use return	No return keyword; the expression's value is returned automatically
Better for complex logic	Better for short, simple logic
Can include loops, multiple conditions, try/except, etc.	Cannot contain statements like for, while, try, or multiple statements

2. What are the limitations of lambda functions?

Answer :

1. Only One Expression (Most Important) :

Square = lambda x: x * x #one line only

2. Cannot Contain Multiple Statements :

Cannot use Below Statements

For, while, try, except, with, match, return, pass, break, continue

3. No return Statement

Right Way : lambda x: x * x

It is wrong :

```
lambda x:
    return x * x
```

4. Not Suitable for Complex Logic

Not Allowed in Lambda function -

```
def Check(Number):
```

```
    if Number % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

5. Cannot Contain Multiple Return Paths Using Statements

Not allowed in Lambda :

```
lambda x:
    if x % 2 == 0:
        return "Even"
    else:
        return "Odd"
```

3. Explain the working of the map() function with an example.

Answer :

The **map()** function applies a given function to **every item of an iterable** (such as a list, tuple, or set) and returns a **map object** (an iterator).

```
def Square(x):
    return x * x

Numbers = [1, 2, 3, 4, 5]

Result = list(map(Square, Numbers))

print(Result)
```

4. How does map() differ from using a for loop?

Answer :

According to the official Python documentation, map(function, iterable, ...) returns an **iterator** that computes the results by applying the given function to each item of the iterable(s). A for loop, on the other hand, is a general-purpose iteration construct that gives the programmer complete control over the processing logic.

map()	for Loop
Applies a function to every element automatically	Processes elements one by one using explicit iteration
Requires a function (or lambda)	Does not require a separate function
Returns a map object (iterator)	Does not return anything automatically
Best for simple transformations	Best for complex logic and multiple statements
Less code	More code
Supports multiple iterables	Requires manual handling for multiple iterables

5. What is the filter() function and when should it be used?

Answer :

filter() selects only those elements that satisfy a specified condition.

Example 1: Using Simple filter function

```
def CheckEven(x):  
    return x % 2 == 0  
  
Numbers = [10, 15, 20, 25, 30]  
  
Result = list(filter(CheckEven, Numbers))  
  
print(Result)
```

Example 2 : Using lambda Function

```
Numbers = [10, 15, 20, 25, 30]  
  
Result = list(filter(lambda x: x % 2 == 0, Numbers))  
  
print(Result)
```

6. Difference between map() and filter().

Answer :

map()

Used to **transform** data

Applies a function to every element

Function returns a transformed value

Returns a **map object**

Output usually has the same number of elements as the input

Best for calculations or modifications

filter()

Used to **filter/select** data

Applies a condition to every element

Function returns True or False

Returns a **filter object**

Output may have fewer elements than the input

Best for selecting required elements

Example 1 : map()

```
Numbers = [1, 2, 3, 4, 5]

Result = list(map(lambda x: x * x, Numbers))

print(Result)
```

Example 2 : filter()

```
Numbers = [1, 2, 3, 4, 5]

Result = list(filter(lambda x: x % 2 == 0, Numbers))

print(Result)
```

7. What is reduce()? Why is it not a built-in function in Python?

Answer :

The **reduce()** function applies a function of **two arguments cumulatively** to the items of an iterable, reducing the iterable to a **single value**.

Unlike map() and filter(), **reduce()** is available in the **functools** module.

Example 1 : reduce using def

```
from functools import reduce
```

```
def Addition(x, y):  
    return x + y  
  
Numbers = [10, 20, 30, 40]  
  
Result = reduce(Addition, Numbers)  
  
print(Result)
```

Example 2 : Using Lambda

```
from functools import reduce  
  
Numbers = [10, 20, 30, 40]  
  
Result = reduce(lambda x, y: x + y, Numbers)  
  
print(Result)
```

8. Explain how **reduce()** works internally.

Answer :

Step 1

Addition(10,20)

↓

30

Step 2

Addition(30,30)

↓

60

Step 3

Addition(60,40)

↓

100

9. Can lambda functions be used with map, filter, and reduce together?

Explain.

Answer :

Yes. Lambda functions can be used with **map()**, **filter()**, and **reduce()** together. This is a common functional programming approach in Python.

- **map()** applies a function to each item of an iterable and returns an iterator.
- **filter()** returns an iterator containing only the elements for which the function returns True.
- **reduce()** (from the `functools` module) applies a two-argument function cumulatively to the items of an iterable, reducing them to a single value.

These functions are designed to work together because **map()** and **filter()** return iterators that can be passed directly to **reduce()** or other iterator-consuming functions.

- ❑ **filter()** → Selects elements based on a condition.
- ❑ **map()** → Transforms the selected elements.
- ❑ **reduce()** → Combines the transformed elements into a single value.

Example :

```
from functools import reduce

Numbers = [10, 15, 20, 25, 30]

Result = reduce(
    lambda x, y: x + y,
    map(
        lambda x: x * x,
        filter(
            lambda x: x % 2 == 0,
            Numbers
        )
    )
)

print(Result)
```

10. When should functional programming be preferred over procedural style in Python?

Answer :

Functional programming should be preferred when your program mainly performs **data transformation, filtering, mapping, or aggregation** using pure functions without modifying shared state.

Example :

```
from functools import reduce

Numbers = [10, 15, 20, 25, 30]

Result = reduce(
    lambda x, y: x + y,
    map(
        lambda x: x * x,
        filter(
            lambda x: x % 2 == 0,
            Numbers
        )
    )
)

print(Result)
```

Procedural programming should be preferred when the program involves **step-by-step** instructions, **complex business logic**, **loops**, **file handling**, **user interaction**, or **state changes**.

Example :

```
Numbers = [10, 15, 20, 25, 30]

Total = 0

for value in Numbers:
    if value % 2 == 0:
        Total += value * value

print(Total)
```